

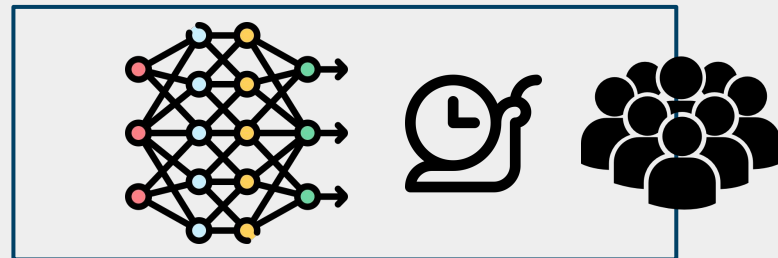
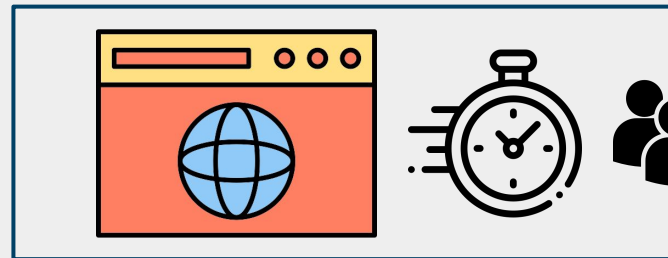
Optimizing LLM Inference: DistServe & Sarathi-Serve

Feb. 12th, 2026

Namita, Nandana, Anika, Joshua

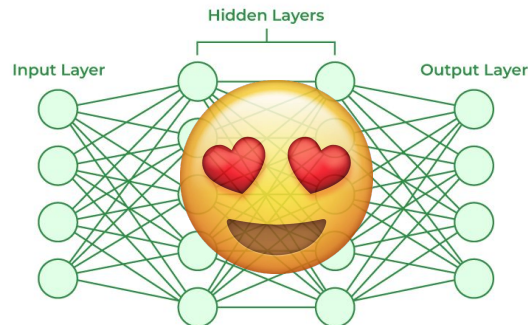
Motivation

- LLMs are reshaping the Internet and have widespread applications
- However, LLM **inference** is slow
- To keep up with demand, providers have to use excess compute
- Both papers want to optimize cost per LLM query - while also prioritizing user experience

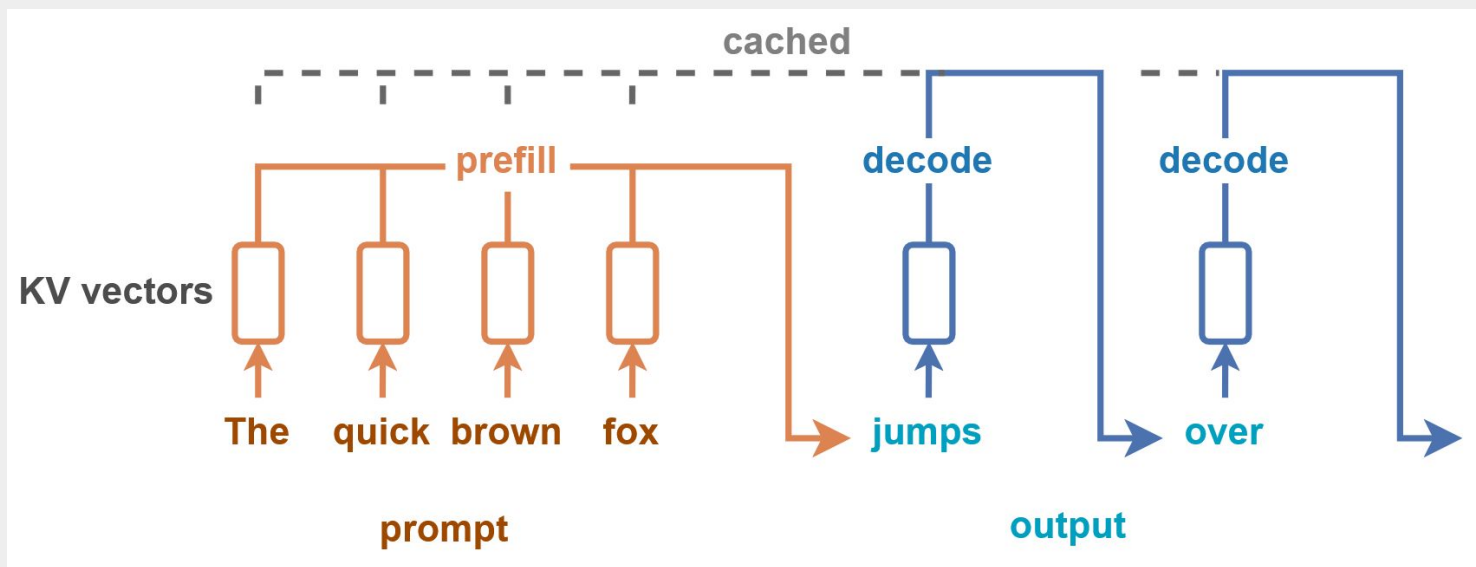


Motivation

- LLMs are reshaping the Internet and have widespread applications
- However, LLM **inference** is slow
- To keep up with demand, providers have to use excess compute
- Both papers want to optimize cost per LLM query - while also prioritizing user experience



LLM Inference is Unique

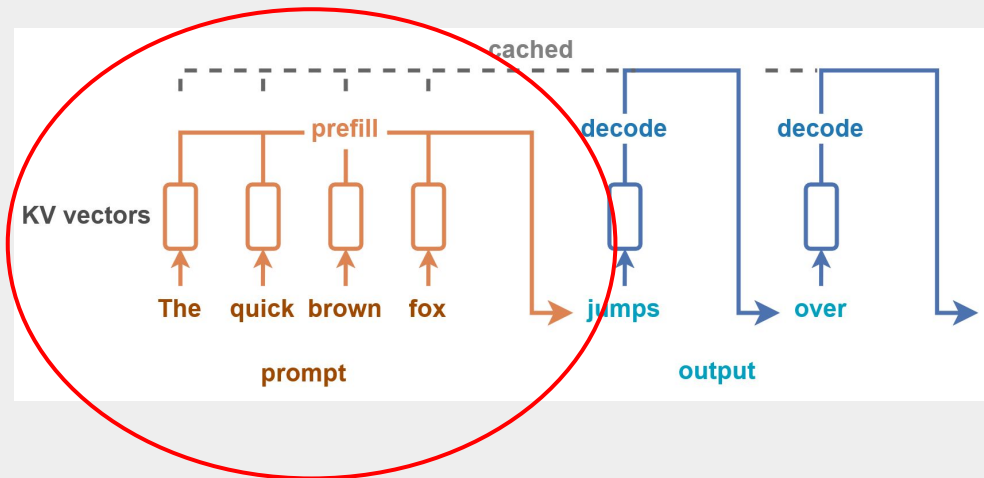


What happens when you send 'Solve my homework' to ChatGPT?

Phase 1: Prefill

Reading and Understanding Stage

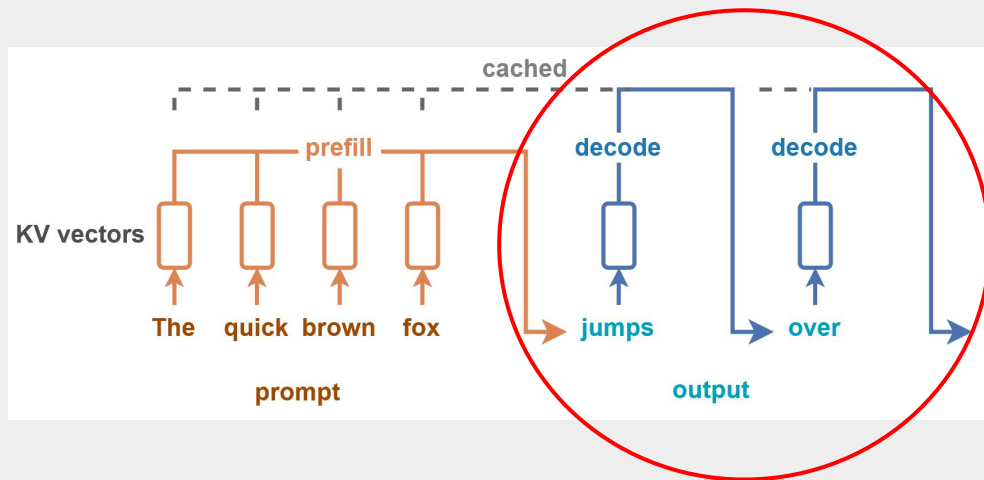
- Goes through entire prompt, **processes all the words**
- Run through transformer layers
- Compute Attention
- Produce first token AND KV Cache.



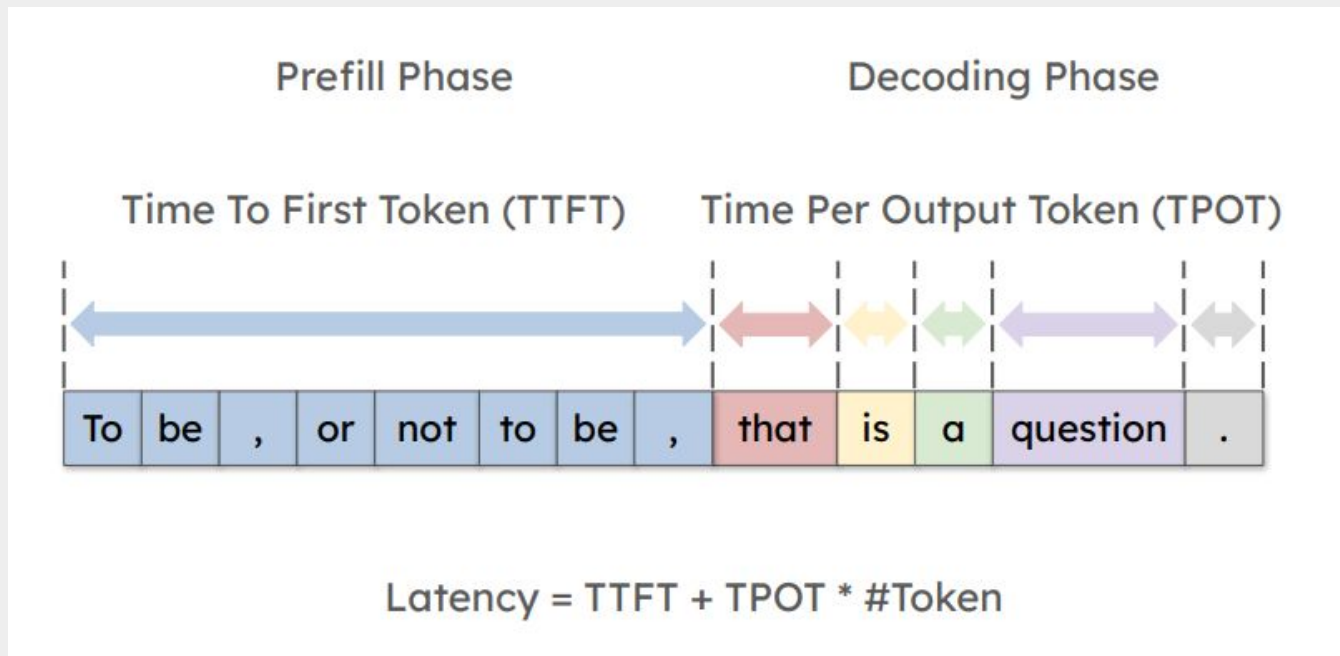
What happens when you send 'Solve my homework' to ChatGPT?

Phase 2: Decode *Writing Stage*

- Autoregressively writes tokens
- Use previous token as input
- Produce **next token probability**
- Append new K and V to KV cache
- Repeat



Metrics for Inference



What happens when you type 'Solve my homework' to ChatGPT?

Prefill : *Reading and Understanding Stage*

- Goes through entire prompt, **processes all the words** - generates the very first word
- Run through transformer layers
- Compute Attention
- Produce first token AND KV Cache.

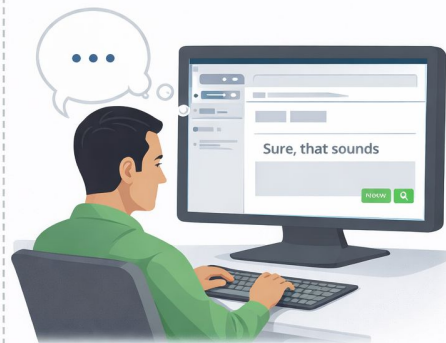
Decode : *Writing Stage*

- Autoregressively writes tokens
- Use previous token as input
- Produce **next token probability**
- Append new K and V to KV cache
- Repeat

Decode Stage

Typing Response One Word at a Time

Carefully Constructing Reply





Compute Bound VS Memory Bound

Prefill : *Compute Bound*

Compute:



Memory:



- During Prefill, we compute Attention - **very intense matrix multiplications!**
- GPU is too busy doing math and is being FULLY UTILIZED.
- Hence → computationally heavy

Do I have enough GPU Compute?

Decode : *Memory Bound*

Compute:



Memory:

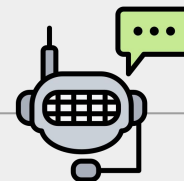
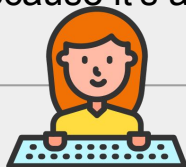


- Less Arithmetic Intensity
- Lot more memory reads - we read KV Cache every time we generate a new token
- Limited by memory bandwidth

How long am I waiting on memory?

Prefill vs Decode

Prefill Phase	Decode phase
Processes user input	Generates LLM output
Metric is TTFT (Time to First Token)	Metric is TPOT (Time per Output Token)
Parallel processing - compute bound	Autoregressive - memory bound
Long time per step	Short time per step, but a lot of steps
Does not benefit from batching (prompt usually saturates GPU because it's arithmetically intensive!)	Benefits from batching



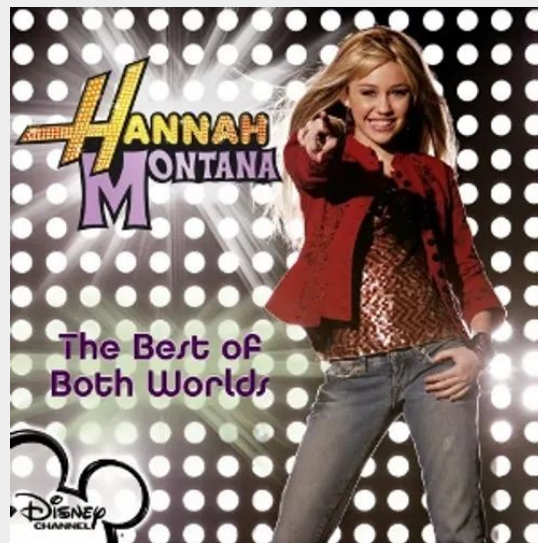
So, what is the Problem?

Observation:

- Prefill = compute-heavy bursts
- Decode = memory-heavy long tail

How do we efficiently serve both stages together?

How do we get the *best of both worlds*?



Previous works

FasterTransformer

May 2022

Orca

June 2023

vLLM

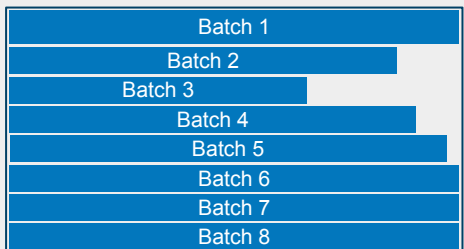
Sept. 2023



Previous Works: Orca & vLLM

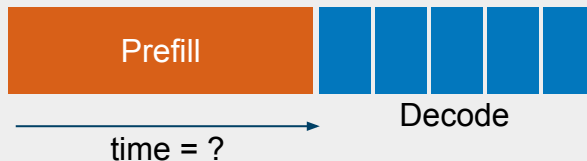
Faster Transformer

- Prioritizes **decoding**
- Limitation: have to wait until all batches are complete to move on to next request



Orca

- Prioritizes **prefill**
- Limitation: prefill can take arbitrarily long, leading to generation stalls



Previous Works: FasterTransformer (NVIDIA)

1. Submit a batch of requests
2. Complete prefill phases
3. Complete all decode phases - no new request can enter



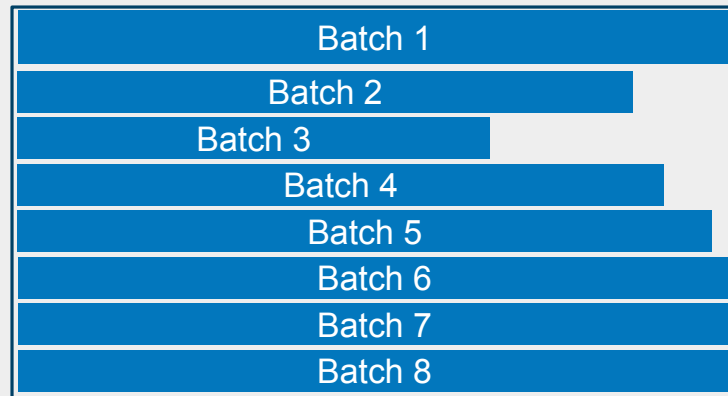
Low **TBT**



Prioritizes **decoding**



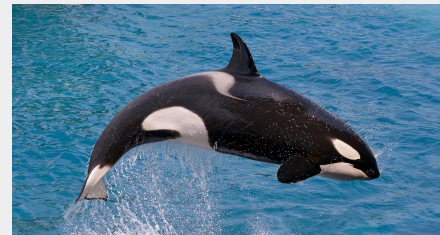
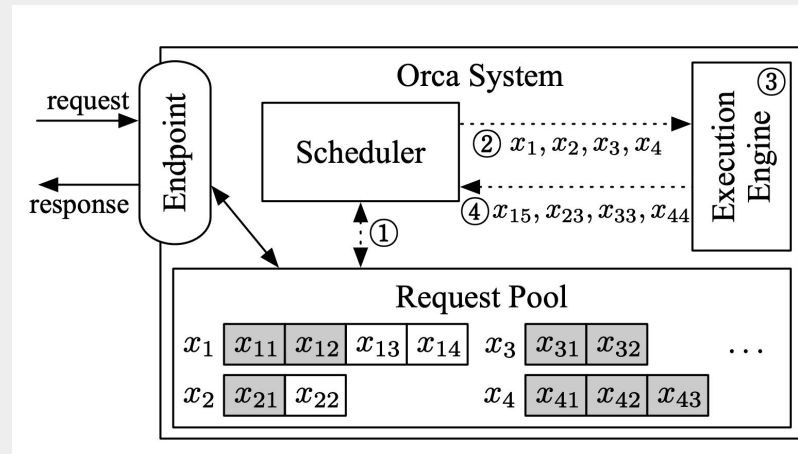
Limited throughput



Previous Works: Orca

1. Instead of batching by request, batch by *iteration*
2. Whenever GPU becomes available, schedule prefill

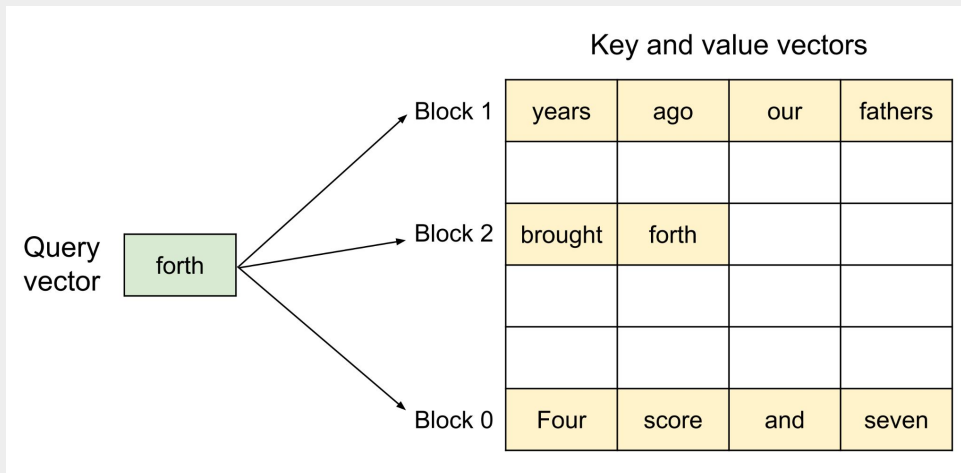
- 😊 Higher throughput
- 😐 Prioritizes **prefill**
- 😞 Generation stalls, memory waste



Previous Works: vLLM

1. Divide KV cache request into blocks
2. During attention computation, fetch relevant blocks separately

- 😊 Flexible memory
- 😐 Still prioritizes **prefill!**
- 😞 Generation stalls





DistServe

**Disaggregating Prefill and Decoding for
Goodput-optimized Large Language Model Serving**

Problem : Currently we Collocate

- Prefill Stage generates KV Cache which we PASS to Decode Stage.
- **Both use the same model weights and KV Cache** → prefill and decode processes are **placed on the same GPU** so Decode directly has access to the cache.

prefill-decode interference

Option 1: Batching Prefill + Decoding Together

Time →

GPU: [PPPPPPPP|D1|D2|D3]

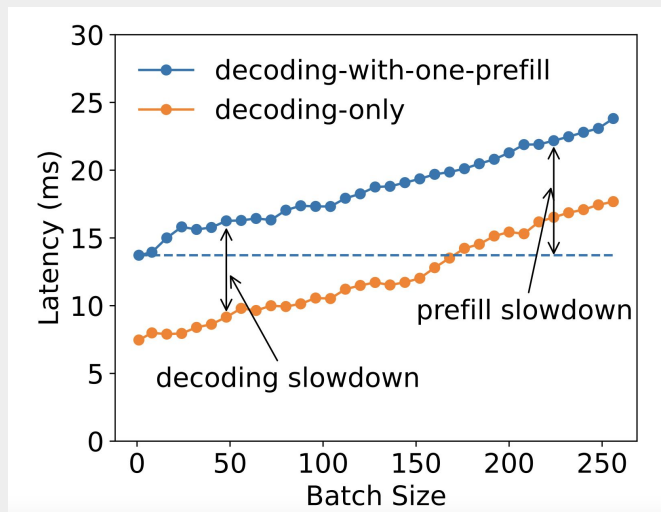
Key Insight:

Decoding slows down because it's waiting for Prefilling to finish.

Prefill slows down because more decoding jobs take up space in the batch.

Current issues

- ☹️ Interference: Adding even 1 prefill job to a batch of decoding requests slows down both phases
 - Potential solution: scheduling?
 - ... Scheduling techniques alone cannot entirely alleviate the interference
 - Prefill and decoding have fundamentally different optimization strategies



Current issues

☹️ Interference: Adding even 1 prefill job to a batch of decoding requests slows down both phases



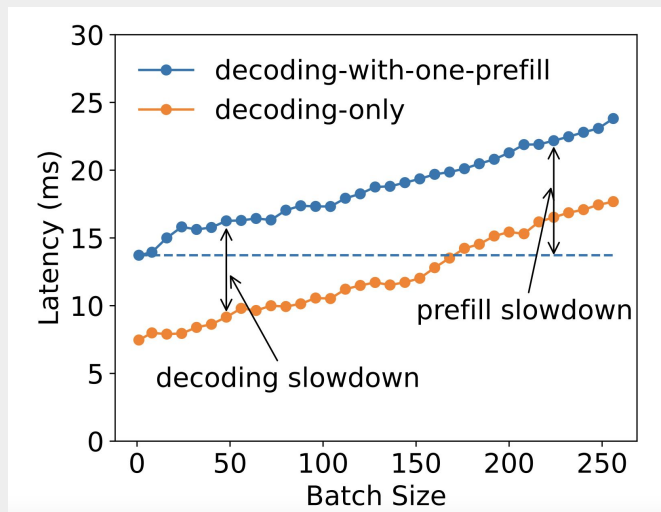
Potential solution: scheduling?



... Scheduling techniques alone cannot entirely alleviate the interference



Prefill and decoding have fundamentally different optimization strategies...



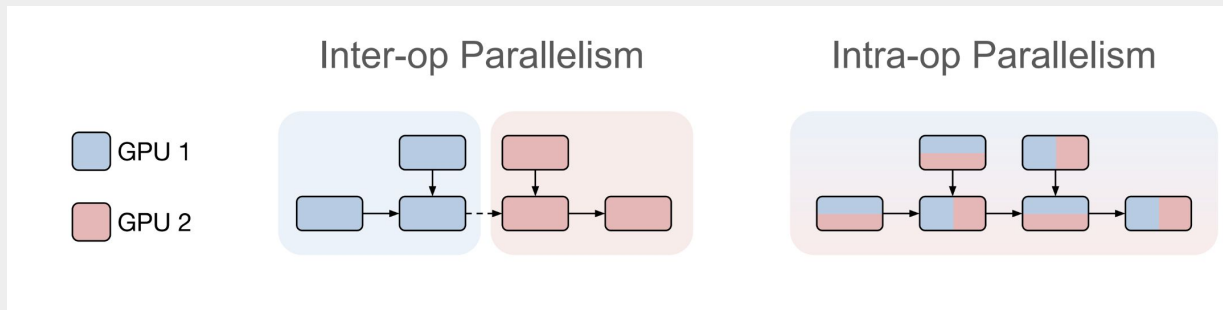
Parallelism

Inter-operator parallelism:

- Assign sequential layers to different devices (pipeline)
- Less frequent communication

Intra-operator parallelism [2]:

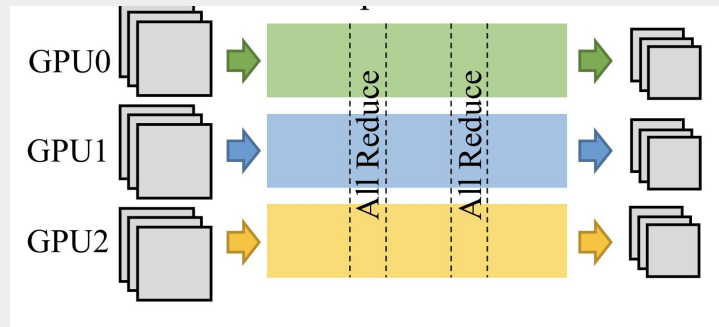
- Partition operators across multiple devices (parallel)
- Frequent communication



Intra-op Parallelism

Intra-operator parallelism:

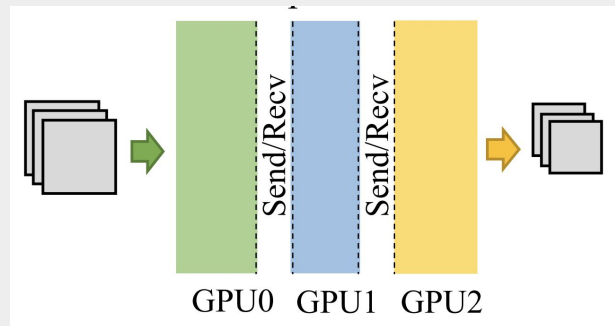
- Partition each operator across multiple devices
- Each device executes a portion in parallel
- Requires frequent communication
- Improves latency, throughput limited



Inter-op Parallelism

Inter-operator parallelism:

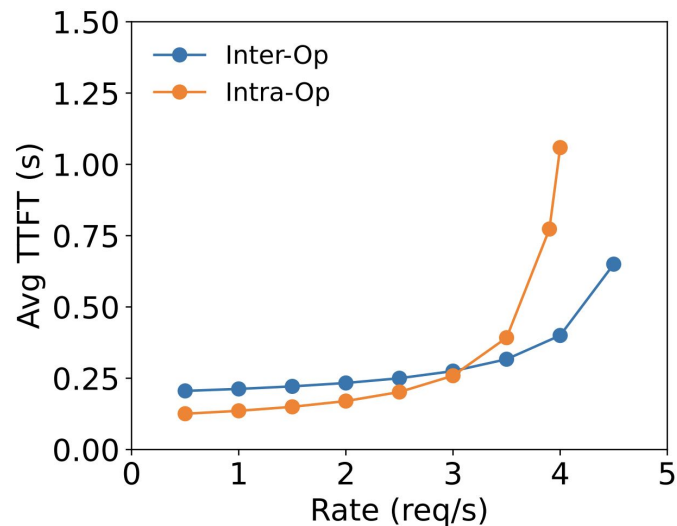
- Assign each stage of the model to a different device
- Requests are processed in a pipeline
- Less frequent communication
- Improves throughput but not rllly latency



Du et al. 2024. *Liger: Interleaving Intra- and Inter-Operator Parallelism for Distributed Large Model Inference*

Prefill instance

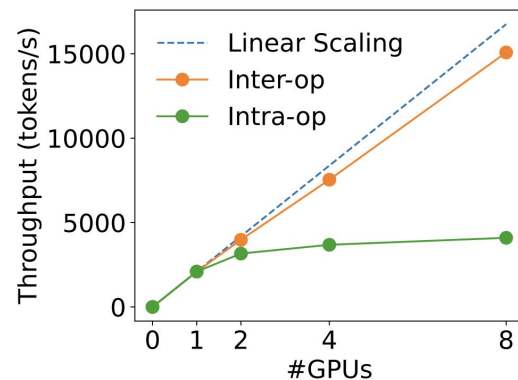
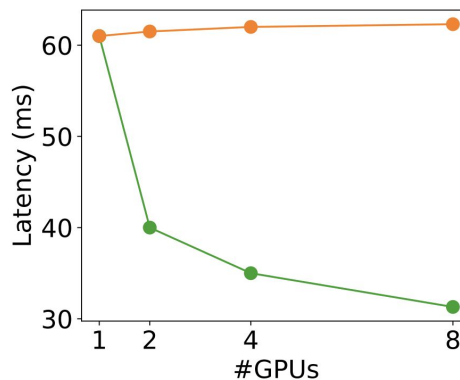
- Intra-op parallelism is favored for lower arrival rates
- Inter-op parallelism is favored for higher arrival rates





Decoding instance

- trade-offs between latency and throughput
- Intra-op parallelism reduces latency with diminishing returns, **caused by communication overhead**



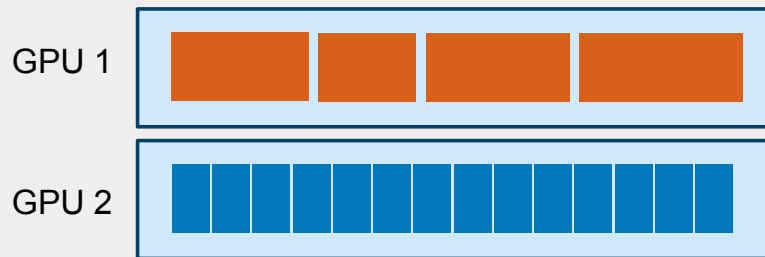
Main idea

- Since prefill and decoding are fundamentally different - why not separate them entirely?
- **Place prefill and decoding on different GPUs**
- Optimize each phase in the most appropriate way for that phase
- How to communicate between phases??????

Before

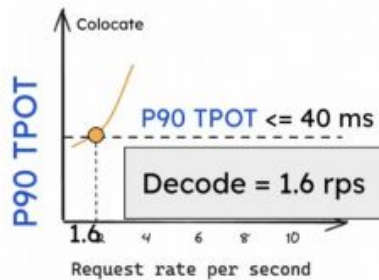
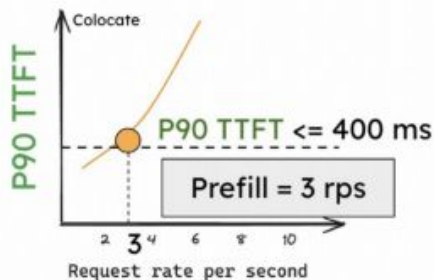


After





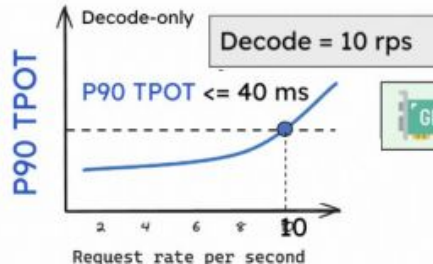
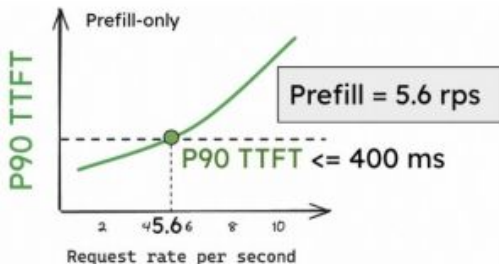
Colocation Versus Disaggregation



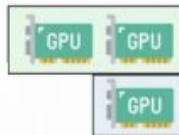
Colocation



Max System rps
 $= \text{Min}(\text{Prefill}, \text{Decode})$
 $= 1.6 \text{ rps} / \text{GPU}$



Disaggregation (2P1D)

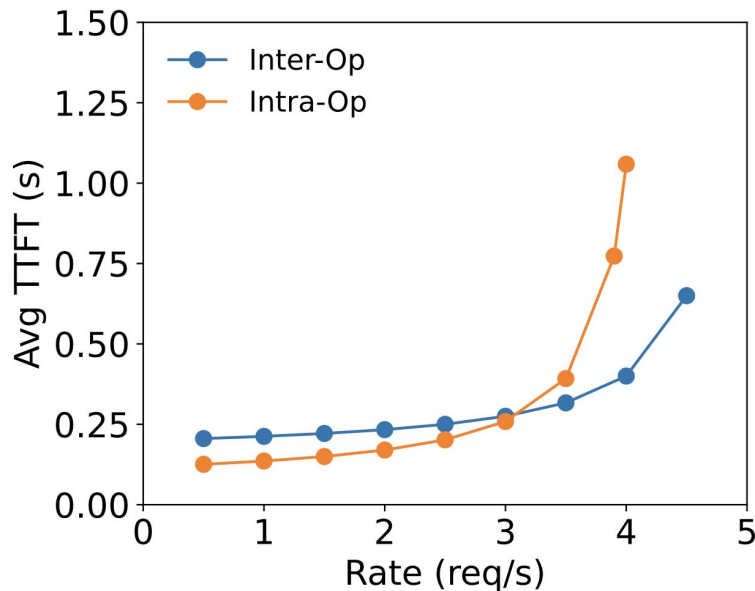


Max System rps
 $= \text{Min}(5.6 \times 2, 10) \text{ rps} / 3 \text{ GPU}$
 $= 3.3 \text{ rps} / \text{GPU}$

Prefill instance

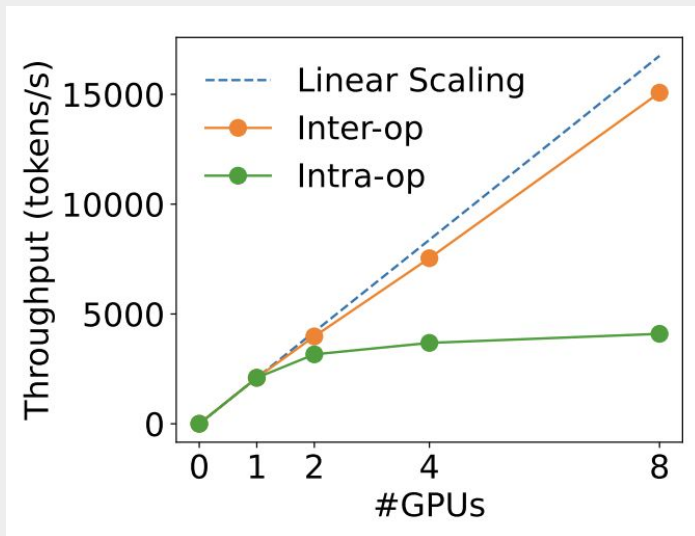
- TTFT = execution time + queuing delay
- Intra-op parallelism is favored for lower arrival rates or tighter SLO
- Inter-op parallelism is favored for higher arrival rates or less strict SLO
- However, the assumption is uniform prompt length... this is addressed later

$$Avg_TTFT = D + \frac{RD^2}{2(1 - RD)}.$$



Decoding instance

- Intra-op parallelism is the winner - since we have a lot of decoding jobs
- Replication is a valid method if the model is small enough (for both prefill and decoding)
- **Big issue:** Transferring the entire KV cache requires a lot of bandwidth!
 - If bandwidth is limited, they use NVLINK → negligible overhead!



Increasing batch size in decoding

Inter-op Parallelism (Pipeline Stages)

- Let's say your model has 60 layers
- **1 GPU (all 60 layers)**
- **2 GPUs (30 layers each)**
- **4 GPUs (15 layers each)**
- Why linear scaling?
 - a. Each GPU becomes a pipeline stage. While GPU-4 finishes batch A, GPU-3 works on batch B, GPU-2 on batch C!
 - b. Effect on latency: Doesn't improve! Each batch still takes time to go through stages.
 - c. Better throughput → less communication between layers

Intra-op Parallelism (Splitting Operations Across GPUs)

- 1 GPU: 100ms latency
- 2 GPUs (intra-op) (not always 50ms!)
- Why diminishing returns?
 - a. Communication overhead: GPUs must synchronize and exchange intermediate results.
- Effect on throughput: Minimal! You're still processing the same batch, just faster.
- Better latency!

Increasing batch size in decoding

- Observation : intra-op parallelism reduces latency with diminishing returns, **caused by communication** and reduced utilization after partitioning.
 - We're trying to split math operations but we need to communicate more in the process!

**Large batch → decode is compute-bound
→ intra-op parallelism reduces latency,
inter-op increases throughput**

The Strategy:

- Stringent TPOT requirements (e.g., must be < 30ms):
- Use intra-op to reduce latency: 1→2→4 GPUs until you meet the 30ms target
- After meeting TPOT, need more throughput:
- Switch to inter-op to linearly scale capacity

DistServe's Contributions

- Find **optimal parallelism strategy** for Prefill/Decoding Instances.
 - In **High Node Affinity Situation** → Nodes are connected with high bandwidth network, e.g., InfiniBand.
 - In **Low Node Affinity Situation** → GPUs inside one node are connected with NVLINK.
- Online Scheduling that adapts to workload in real time

Goal : Use the **fewest #GPUs** to serve **as many requests as possible** while subject to **various latency requirements** of apps.

aka Maximize per GPU Goodput

High Node Affinity Design

- KV Cache transmission overhead is negligible (these are REALLY FAST)
- We can optimize for Prefill and Decode **separately** in this case → then use replication.
- **New Problem : Find Optimal Parallel Configuration for SINGLE instance** (prefill/decode) that meets your SLO => % of reqs that meet TTFT requirement (not an easy task!)

High Node Affinity Design

Solution : Build a Simulator

1. Learn workload pattern :
rates, input/output length
(from historical data)
2. Generate a synthetic
workload based on patterns
3. Simulate each configuration
 - a. Enumerate parallel
configurations
 - b. Find maximum
goodput via Binary
Search
4. Find best configuration!

for every possible

if config memory
less than GPU
capacity

simulate prefill
and decode and
find max goodput
via binary search

Algorithm 1 High Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

```

 $config_p, config_d \leftarrow \emptyset, \emptyset$ 
for  $intra\_op \in \{1, 2, \dots, M\}$  do
  for  $inter\_op \in \{1, 2, \dots, \frac{N \times M}{intra\_op}\}$  do
    if  $\frac{G.size}{inter\_op \times intra\_op} < C$  then
       $config \leftarrow (inter\_op, intra\_op)$ 
       $\hat{G} \leftarrow parallel(G, config)$ 
       $config.goodput \leftarrow simu\_prefill(\hat{G}, W)$ 
      if  $\frac{config_p.goodput}{config_p.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
         $config_p \leftarrow config$ 
       $config.goodput \leftarrow simu\_decode(\hat{G}, W)$ 
      if  $\frac{config_d.goodput}{config_d.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
         $config_d \leftarrow config$ 
 $n, m \leftarrow \lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$ 
 $best\_plm \leftarrow (n, config_p, m, config_d)$ 
return  $best\_plm$ 
    
```

Low Node-Affinity Design

- **Key Insight** : KV Cache transmission occurs between corresponding **LAYERS** of **prefill** and **decode**.
- Collocate prefill and decode segments of SAME layers (Groups of Stages) → intermediate transfer can now occur **WITHIN** the node which is **FAST**
- **Prefill Stage 0** only needs to send **KV cache to Decode Stage 0**, not to Decode Stage 1 or 2.

NODE A:

Time 100ms: Prefill Seg 0 finishes layers 1-20

→ IMMEDIATELY transfers KV cache to Decode Seg 0 (same node, NVLINK!)

NODE B:

Time 200ms: Prefill Seg 1 finishes layers 21-40

→ IMMEDIATELY transfers KV cache to Decode Seg 1 (same node, NVLINK!)

Evaluation

Setup:

- 4 nodes, 32 GPUs : 8 NVIDIA SXM A10080GB GPUs connected with NVLINK.
- Cross node bandwidth = 25GBps
- Low bandwidth for cross node → low node affinity algorithm

Workloads :

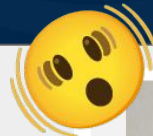
Metric : SLO

Baseline :

- vLLM - supports
- DeepSpeed - MII interference but c

Loose TTFT, tight TPOT

Higher than normal human read speed!



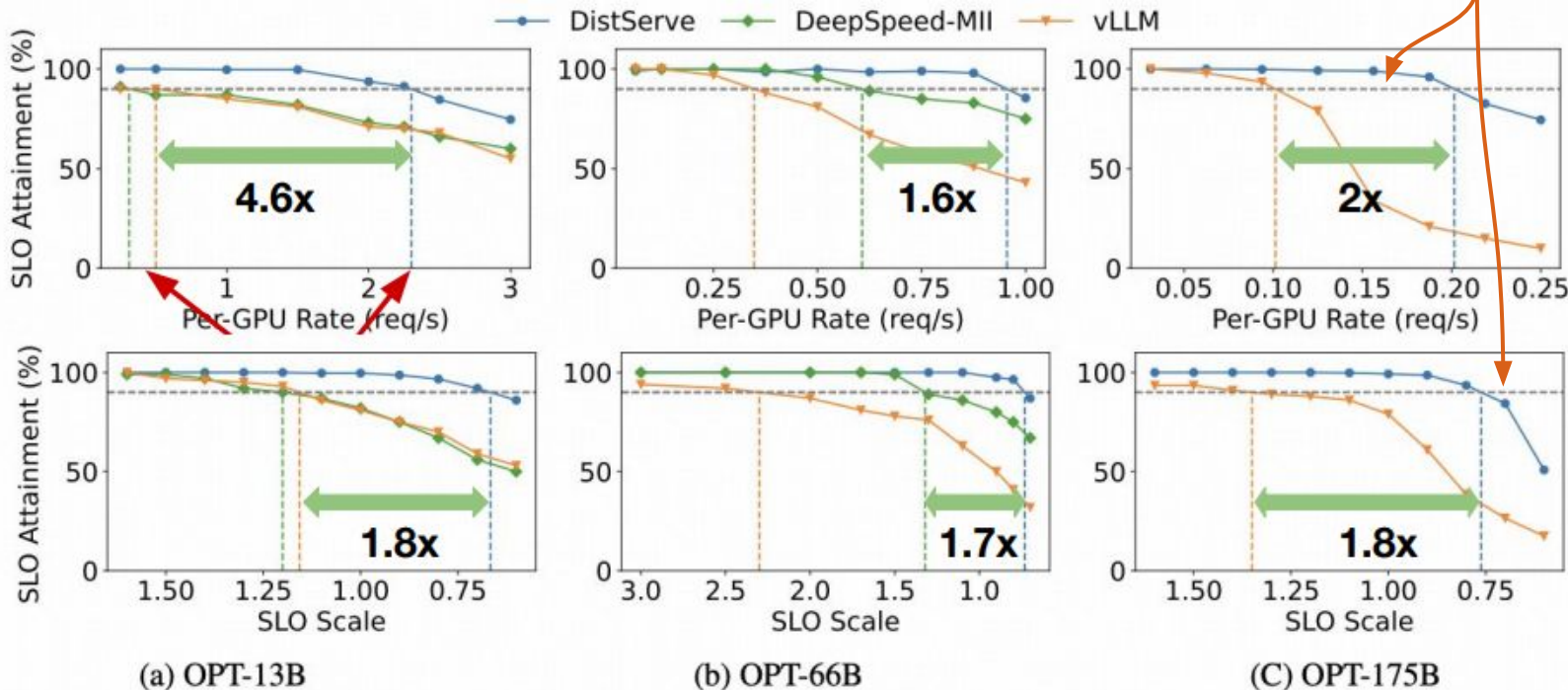
ENDED
Nvidia A100 80GB
SXM4 Graphics Card
Tensor Core GPU
699-2G506-0210-320
US \$5,250.00
Mon, Feb 09, 03:09 AM
[See original listing](#)

Application	Model Size	TTFT	TPOT	Dataset
Chatbot OPT-13B	26GB	0.25s	0.1s	ShareGPT [8]
Chatbot OPT-66B	132GB	2.5s	0.15s	ShareGPT [8]
Chatbot OPT-175B	350GB	4.0s	0.2s	ShareGPT [8]
Code Completion OPT-66B	132GB	0.125s	0.2s	HumanEval [14]
Summarization OPT-66B	132GB	15s	0.15s	LongBench [13]

Table 1: Workloads in evaluation and latency requirements.

maximum per-GPU rate the system can handle to meet latency requirements for over 90% of the requests

Evaluation : Chatbot Results

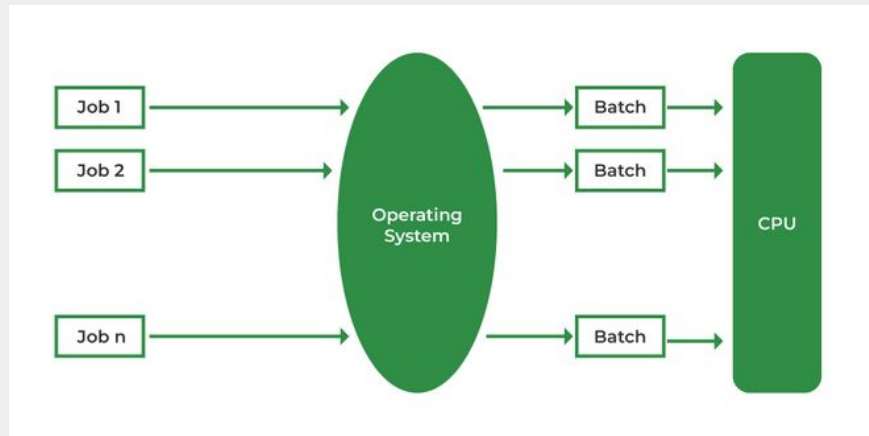


Sarathi-Serve

**Taming Throughput-Latency Tradeoff
in LLM Inference with Sarathi-Serve**

Batching

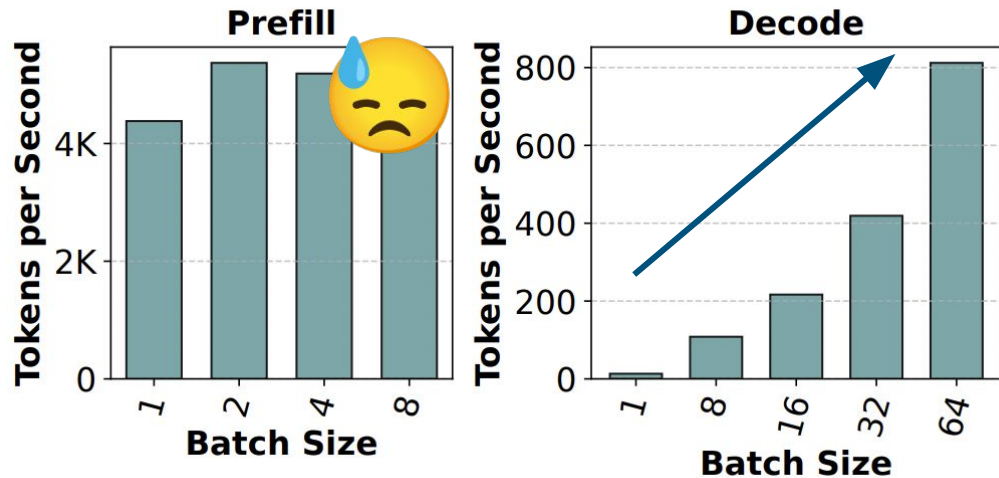
- Processing multiple requests together in a single forward pass instead of one at a time.
- **Increasing** Batch Size =
 - ↑ the number of tokens processed per iteration.
 - ↑ arithmetic intensity.
 - ↑ GPU utilization.
 - ↑ Throughput (tokens per second)





Current issues

How can we we maintain
low latency
with **high throughput?!**





Main Idea - Balanced Hybrid Batches

IDEA:

Fused computation of prefill and decodes that reduces bubbles

HOW?

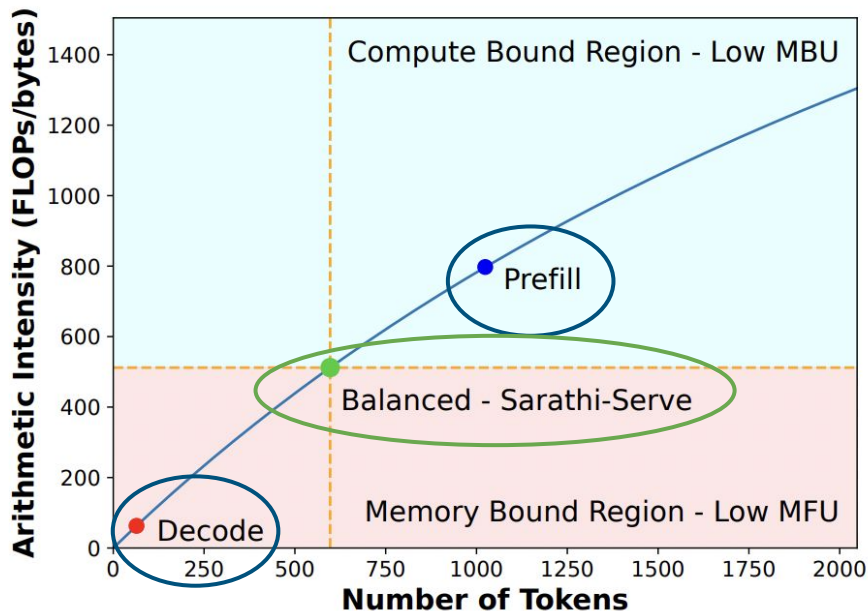
Mix decode tokens with chunked prefill tokens using **chunking** and **stall-free scheduler!**



BUT!
Can't do it
randomly!

Why This Works?

- Interleaving:
 - Balances hardware pressure
 - Improves arithmetic intensity
 - Increases GPU utilization





Design - Chunked Prefill

- Break prefill into smaller chunks instead of processing full prompt at once
- Enables fine-grained scheduling
- Can fit into decode bubble windows

BUT.....How do you choose the right chunk size?

- Too large → cannot fit into pipeline gaps
- Too small → scheduling overhead increases
- Sarathi chooses based on
 - **pipeline's token budget**
 - **decode cadence**



Design - Stall-Free Scheduling

- Dynamically schedules:
 - Decode tokens
 - Prefill chunks
- Ensures no stage goes idle
- Maintains continuous pipeline execution



Design - Stall-Free Scheduling

BUT.....HOW?

Algorithm:

1. Always admit all ongoing decodes into the batch.
2. Add partially completed prefills (chunked).
3. Admit new requests if token budget allows.

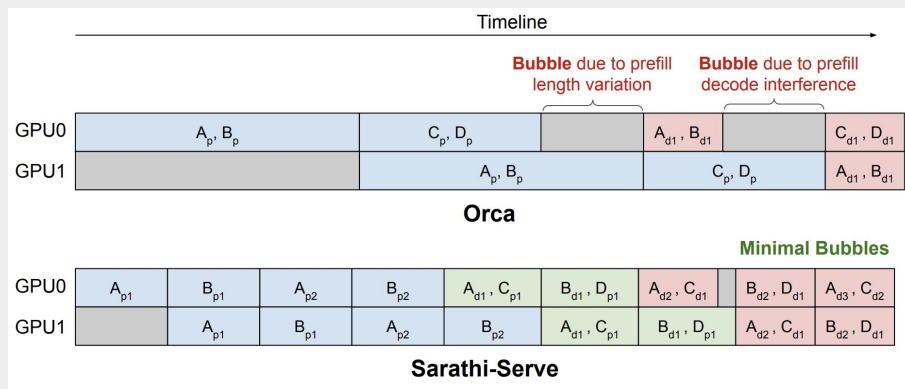
Token Budget

- Total pipeline capacity per scheduling iteration
- Decode tokens are admitted first (latency priority)
- **Remaining capacity = Token Budget**
 - Safe slack in the pipeline
 - Maximum additional tokens that can be inserted without delaying decode



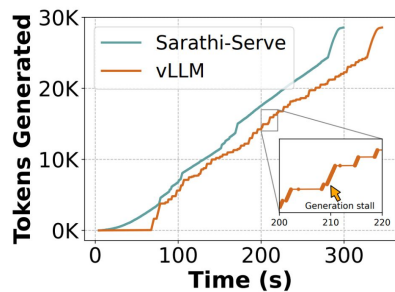
Why is this better than the competition?

- Currently, continuous batching:
 - Improves memory efficiency
 - Reduces fragmentation
- But:
 - Does not eliminate pipeline bubbles
- Sarathi:
 - Fixes pipeline-level inefficiency!!**

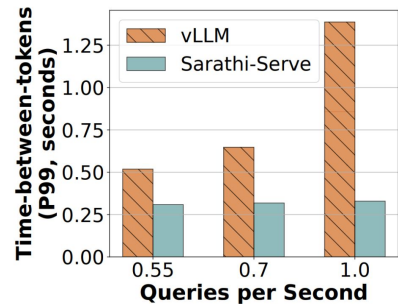


Evaluation of Sarathi-Serve

- Multiple open weight models (7B-180B) and GPU configurations (1-8 GPUs, 1-2 nodes)
- Two open datasets
 - openchat_sharegpt4
 - arxiv_summarization (longer context)
- Compared against vLLM and Orca
- Objectives:
 - Maximum load under SLO constraints
 - Effect of parallelism (TP, PP)
 - Chunked prefill overhead
 - Ablation study of features
- Details released for reproducibility



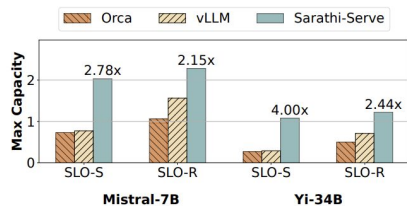
(a) Generation stall.



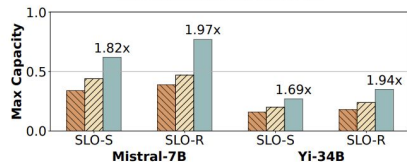
(b) High tail latency.

Workload Evaluation

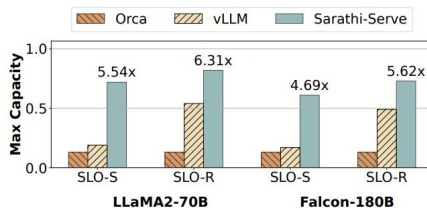
- SLO: Model- and hardware-specific time between tokens objective
 - Strict: 5x decode time
 - Relaxed: 25x



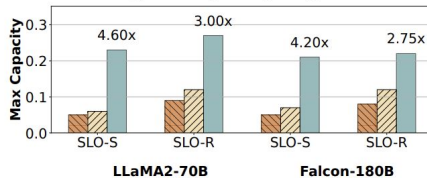
(a) Dataset: *openchat_sharegpt4*.



(b) Dataset: *arxiv_summarization*.



(a) Dataset: *openchat_sharegpt4*.



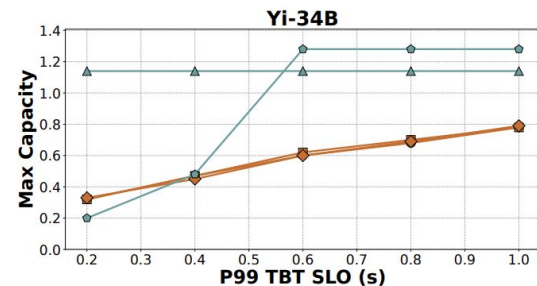
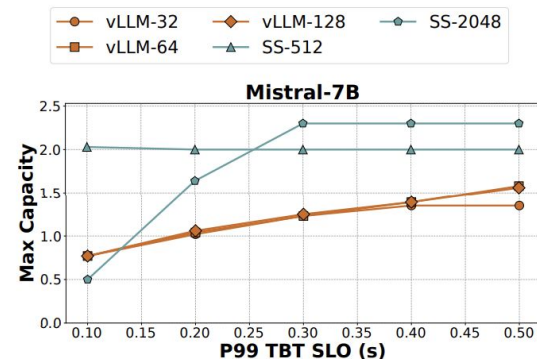
(b) Dataset: *arxiv_summarization*.

Model	<i>relaxed</i> SLO P99 TBT (s)	<i>strict</i> SLO P99 TBT (s)
Mistral-7B	0.5	0.1
Yi-34B	1	0.2
LLaMA2-70B	5	1
Falcon-180B	5	1

Table 3: SLOs for different model configurations.

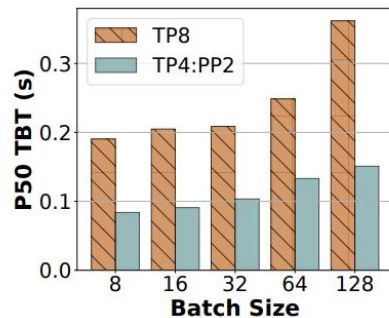
Throughput-Latency Tradeoff

- P99 TBT latency SLO, strict/relaxed
- vLLM generation stalls limit capacity
 - Prefill-prioritizing scheduler
- Sarathi-Serve varies token budget to increase capacity

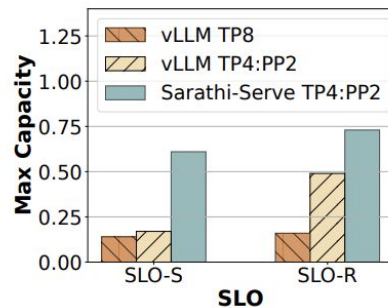


Parallelism

- vLLM w/ Tensor Parallelism scales poorly
 - High communication overhead
 - High latency -> low capacity
 - Performance drops in strict setting - pipeline bubbles
- Sarathi-Serve: chunked prefills reduce execution time variation -> capacity increase
 - (4-way TP within node, 2-way PP across nodes)
 - 1.48x relaxed capacity increase
 - 3.6x strict capacity increase



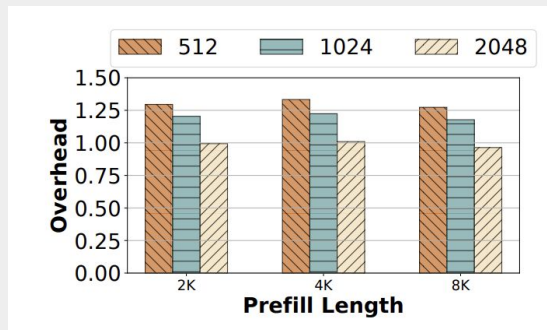
(a) TBT (Falcon-180B).



(b) Capacity (Falcon-180B).

Ablation Study

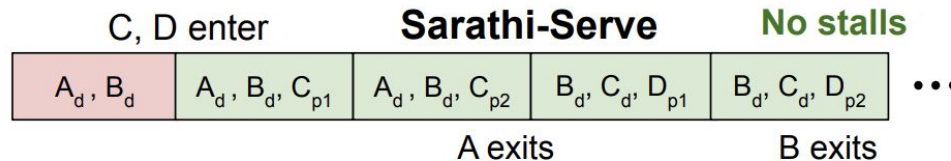
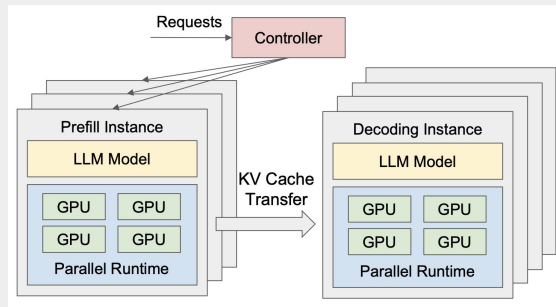
- Chunked prefill overhead on runtime:
 - Smaller chunks, higher overhead
 - At most ~25%
- Chunked prefills and hybrid batching work best together
 - Chunked prefills: Prefill chunks are slightly inefficient
 - Hybrid batching: Long prefills can still create generation bubbles



Scheduler	openchat_sharegpt4		arxiv_summarization	
	P50 TTFT	P99 TBT	P50 TTFT	P99 TBT
hybrid-batching-only	0.53	0.68	3.78	1.38
chunked-prefills-only	1.04	0.17	5.38	0.20
Sarathi-Serve (combined)	0.76	0.14	3.90	0.17

DistServe vs. Sarathi-Serve

- DistServe can eliminate prefill-decode interference
- DistServe achieves better TTFT: prefills can be executed with maximum efficiency
- Sarathi-Serve achieves higher throughput: bubbles are removed
- Disaggregation requires KV cache migration after prefill: communication bottleneck





DistServe vs. Sarathi-Serve in Practice

DistServe

- Better for applications with strict latency requirements
- Better for more GPUs with dedicated clusters for prefill and decode
- Potential application: large-scale chatbot deployment with strict latency requirements

Sarathi-Serve

- Better when balancing latency, throughput in an application
- Better in cost-constrained, lower-compute applications
- Potential application: smaller-scale, cost efficient interactive applications with variable traffic/workloads

Caveat: Evaluation Flaws

On Evaluating Performance of LLM Inference Systems [1]

- Agrawal et al. argue some evaluations aren't representative of true performance

DistServe:

- vLLM, DistServe, DeepSpeed-MII use different codebases
- Parameters like chunk, batch sizes not documented in certain cases
- Limited to short-context scenarios (max prefill of 2k, decode of 200)

Sarathi-Serve:

- 5s TBT P99 bounds in "relaxed" setting not practical
- P99 TBT extensively covered, median not

References

- [1] Amey Agrawal, Nitin Kedia, Anmol Agarwal, Jayashree Mohan, Nipun Kwatra, Souvik Kundu, Ramachandran Ramjee, and Alexey Tumanov. 2025. On Evaluating Performance of LLM Inference Serving Systems. Retrieved from <https://arxiv.org/abs/2507.09019>
- [2] Du, Wei, Jiang, Cheng, Huang, Chen, Lu. 2024. Liger: Interleaving Intra- and Inter-Operator Parallelism for Distributed Large Model Inference. Retrieved from <https://dl.acm.org/doi/pdf/10.1145/3627535.3638466>

Margins

Keep the bottom and sides of your slides clear.

Universal Design benefits all users, and whitespace improves readability.

Protect the bottom margin.

Closed captioning on Zoom will appear at the bottom of the screen.

Keep the bottom margin clear to avoid overlapping your images or text. The teal arrows at the bottom of these slides mark off the captioning area.

Zoom captions appear here:

"Good morning, and welcome to my session on the importance of..."

Typography

Keep fonts as large as possible.

Use no more than 2 fonts, and use **bold**, *italics*, CAPS sparingly.

Use appropriate contrast combinations.

As far as font size goes, go big.

In this template, the default font sizes

- title 48pt
- subtitle 34pt
- body text 16pt

Generally speaking, don't go smaller than

- subtitle text 24pt
- body text 16pt

As a rule, if your options are to squeeze more text onto a slide or add another slide, **add the extra slide.**

Font styles - also keep it simple.

- Use no more than two fonts, and use them consistently
- Use bold, italics, and ALL-CAPS sparingly, also consistently

Google offers a million *FUN, dramatic, moody* fonts. Use these sparingly, if at all. Use web safe fonts and learn about typeface readability.

Web safe fonts are not only the simplest, but they are also the most consistent – available across all operating systems and devices.

Images (and graphics)

- Large and distinct.
- Color can't be the only indicator of meaning.
- Alt-Text to describe each image.
- Avoid screenshots of text, if possible.

Feature every image.

Make images **LARGE**

- **One per slide** works best, unless side-by-side comparison is critical.
- Too many images can trigger **cognitive overload**.
- The more images per slide, the smaller and harder to see each becomes.
- Remember to reserve the bottom of each slide for captions.
- Use **high-resolution** images.

Avoid screenshots of text.

- Screen readers can't read them
- Text doesn't resize cleanly
- Hard to correct for contrast

If you must, be sure to describe the text in alt-text.

Reading order

Use the tab key to move through each slide and confirm that elements will be read in the right order.

The title should always come first

In this two-column layout, this column would be read next.

...and this column would be read last. If they're out of order, use the "order" option under Arrange to make adjustments.

Important note: In Google slides, "Send to back" will cause an element to be read **first**.